

Science des données

Ce TP est dédié à l'utilisation de techniques de **classification supervisée**. Contrairement aux algorithmes de *clustering* vus lors des séances précédentes, qui permettent de réaliser des **classifications non supervisées**, ces techniques apprennent à classer à partir d'un premier jeu de **données étiquetées**, dans lequel les étiquettes représentent les classes. Ils peuvent ensuite étiqueter de nouvelles données, non étiquetées au préalable cette fois, en utilisant les règles de classement apprises pendant la phase d'apprentissage. Le TP comporte 3 exercices. Le premier porte sur la classification d'aliments, le second porte sur la classification de contes, le troisième porte sur la classification d'iris.

Mots-clés : classification supervisée, classification non supervisée, données étiquetées, validation croisée, matrice de confusion, prédiction, nomogramme, classification binaire, classification multiclasses.

Exercice 1

Ouvrez un nouveau fichier *Orange* et importez le jeu de données *fruits-and-vegetables-train.tab* disponible sur Moodle grâce à l'outil *File*. Il contient 35 individus avec :

- 1 variable textuelle (*name*) contenant le nom d'un fruit ou d'un légume,
- 9 variables quantitatives caractérisant le fruit ou le légume,
- 1 variable nominale (*classification*) indiquant si l'individu est un fruit (*fruit*) ou un légume (*vegetable*).

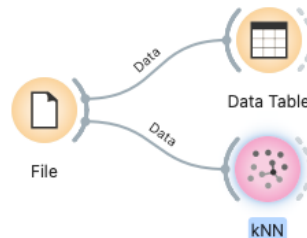
Columns (Double click to edit)				
	Name	Type	Role	Values
1	vitamin A %	N numeric	feature	
2	vitamin C %	N numeric	feature	
3	calcium %	N numeric	feature	
4	iron %	N numeric	feature	
5	magnesium %	N numeric	feature	
6	calories (per ...	N numeric	feature	
7	potassium ...	N numeric	feature	
8	protein (g)	N numeric	feature	
9	fiber (g)	N numeric	feature	
10	classification	C categorical	target	fruit,vegetable
11	name	S text	meta	

Ajoutez *Data Table* en sortie de *File* et parcourez les données de façon à bien les comprendre :

Info 35 instances (no missing data) 9 features Target with 2 values 1 meta attribute										
Variables ☒ Show variable labels (if present) ☒ Visualize numeric values ☒ Color by instance classes										
Selection ☒ Select full rows										
		classification	name	vitamin A %	vitamin C %	calcium %	iron %	magnesium %	calories (per 100g)	potassium (mg)
1	fruit	apple	1.0	7.0	0.0	0.0	1.0	52.0		
2	fruit	apricot	38.0	16.0	1.0	2.0	2.0	48.0		
3	fruit	avocado	2.0	16.0	1.0	3.0	7.0	160.0		
4	fruit	banana	1.0	14.0	0.0	1.0	6.0	89.0		
5	vegetable	beetroot	0.0	8.0	1.0	1.0	5.0	43.0		
6	fruit	blackberry	4.0	35.0	2.0	3.0	5.0	43.0		
7	fruit	blueberry	1.0	16.0	0.0	1.0	1.0	57.0		
8	vegetable	broccoli	12.0	148.0	4.0	3.0	5.0	34.0		
9	vegetable	brussels sprouts	15.0	141.0	4.0	7.0	5.0	43.0		
10	vegetable	carrot	334.0	9.0	3.0	1.0	3.0	41.0		

L'objectif de cet exercice est d'entraîner des modèles à reconnaître, à partir des variables quantitatives, l'étiquette contenue dans la variable *classification*. Autrement dit, vous cherchez à créer des modèles classant les individus en fruits ou en légumes. Une fois entraînés, vous allez utiliser les modèles sur un jeu de données ne contenant pas la variable *classification*.

Pour commencer, ajoutez l'algorithme des *k* plus proches voisins (outil *kNN* dans le menu *Model*) en sortie de *File* :



Ouvrez *kNN* :

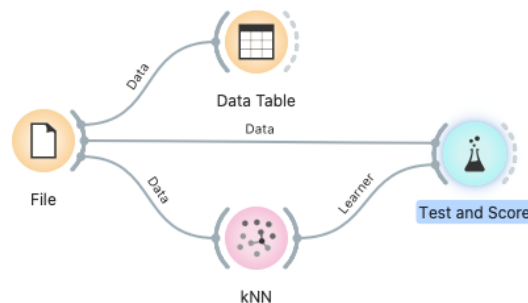
The screenshot shows the configuration window for the 'kNN' model. It has a 'Name' field containing 'kNN'. Below it, the 'Neighbors' section has a 'Number of neighbors' set to 5. The 'Metric' is set to 'Euclidean' and the 'Weight' is set to 'Uniform'. Both dropdown menus have blue arrows indicating they can be changed.

Comme vous pouvez l'observer, vous pouvez modifier 3 paramètres :

- *Number of neighbours* : le nombre *k* de plus proches voisins,
- *Metric* : la mesure de distance utilisée,
- *Weight* : le poids des plus proches voisins (si la distance est sélectionnée, les individus les plus proches, parmi les *k* plus proches voisins, de l'individu à classer compteront plus que les plus éloignés).

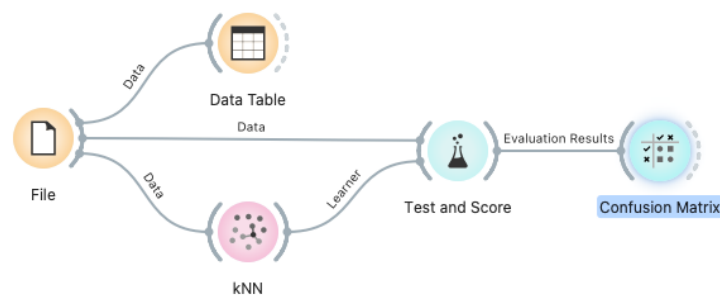
Pour l'instant, laissez les paramètres par défaut tels qu'indiqués dans l'image ci-dessus.

En sortie de *kNN*, ajoutez l'outil *Test and Score* (menu *Evaluate*). En plus du *kNN*, il doit aussi prendre en entrée les données issues de *File* :



Il permet de générer une **validation croisée** du modèle. Ouvrez-le et observez les différents paramètres sur la gauche. En particulier, vous pouvez modifier le nombre de blocs (*Number of folds*) utilisés. Mettez le nombre de blocs à 10. Sur la droite, vous voyez apparaître différentes valeurs mesurant la qualité du modèle (*CA*, *F1*, *Precision*, *Recall*). Nous y reviendrons la semaine prochaine.

En sortie de *Test and Score*, ajoutez une **matrice de confusion** (*Confusion Matrix*, menu *Evaluate*) :



Ouvrez la matrice de confusion :

		Predicted		
		fruit	vegetable	Σ
Actual	fruit	18	2	20
	vegetable	7	8	15
Σ		25	10	35

Vous pouvez observer que 18 fruits et 8 légumes ont été correctement classés. Par contre, 7 légumes ont été classés comme des fruits et 2 fruits ont été classés comme des légumes. Le modèle est donc plutôt bon mais loin d'être infaillible.

Modifiez les paramètres de *kNN* jusqu'à obtenir la matrice de confusion suivante (c'est la meilleure que l'on puisse obtenir avec l'algorithme des *k* plus proches voisins) :

		Predicted		
		fruit	vegetable	Σ
Actual	fruit	20	0	20
	vegetable	5	10	15
Σ		25	10	35

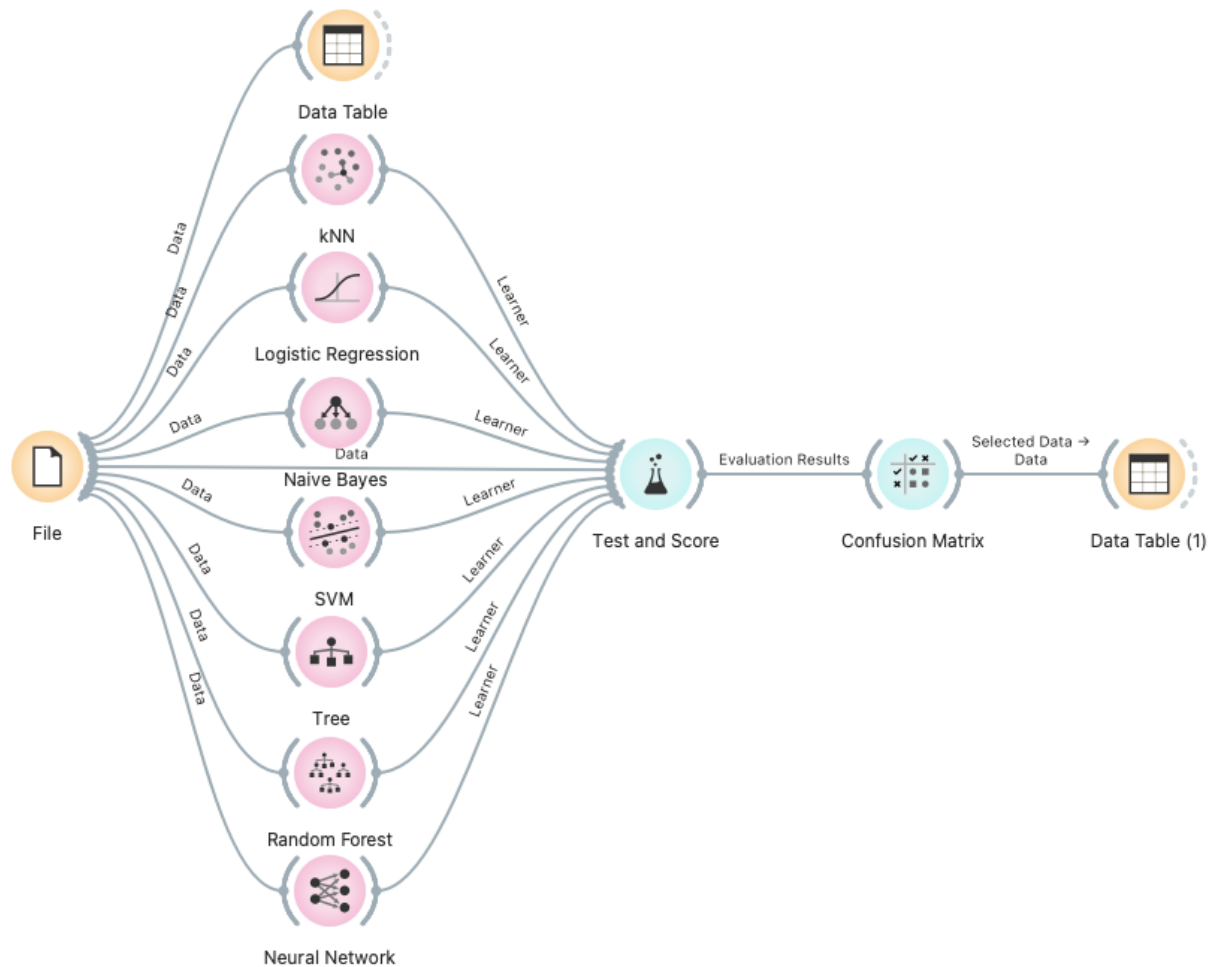
Ajoutez un *Data Table* en sortie de *Confusion Matrix*. Ouvrez-le. Ouvrez aussi *Confusion Matrix* et sélectionnez les 5 individus classés comme fruits alors que ce sont des légumes en cliquant sur la case correspondante. Vous pouvez alors observer ces individus dans la table :

		Predicted		
		fruit	vegetable	Σ
Actual	fruit	20	0	20
	vegetable	5	10	15
Σ		25	10	35

	classification	name	classification(kNN)	vitamin A %	vitamin
1	vegetable	cucumber	fruit	2.0	.
2	vegetable	pepper	fruit	7.0	_____
3	vegetable	leek	fruit	33.0	_____
4	vegetable	watermelon	fruit	11.0	_____
5	vegetable	eggplant	fruit	0.0	_____

Il s'agit du concombre, du poivre, du poireau, de la pastèque et de l'aubergine. En ce qui concerne la pastèque, on peut se demander si la classification initiale était correcte...

Vous allez maintenant comparer *kNN* avec les autres modèles de classification évoqués en cours. Ajoutez les modèles disponibles dans le menu *Model* (*Logistic regression*, *Naive Bayes*, *SVM*, *Tree*, *Random Forest*, *Neural Network*) :



Ouvrez *Matrix Confusion* et comparez les différentes matrices de confusion. Quel modèle fait le moins d'erreurs et paraît donc être le plus performant ?

		Predicted		
		fruit	vegetable	Σ
Actual	fruit	19	1	20
	vegetable	3	12	15
Σ		22	13	35

Remarque : Vous avez modifié les paramètres de *kNN* pour en améliorer les résultats. Les autres modèles ont aussi des paramètres. Tout au long de votre formation, vous allez étudier chacun de ces modèles pour arriver à comprendre l'impact de leurs paramètres et ainsi être capable de les modifier pour améliorer les résultats.

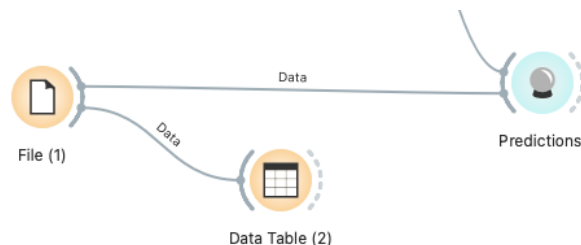
Maintenant que vous avez comparé vos modèles et identifié celui qui produit les meilleurs résultats, vous allez pouvoir faire des **prédictions**. Pour cela, commencez par récupérer le fichier

fruits-and-vegetables-test.tab sur Moodle, chargez-le dans *Orange* à l'aide d'un second *File* et visualisez son contenu à l'aide d'un *Data Table* :

	name	vitamin A %	vitamin C %	calcium %	iron %	magnesium %	calories (per 100g)	potassium (mg)	protein (g)	fiber (g)
1	?	1.0	154.0	3.0	1.0	4.0	61.0	213.0	1.1	3.0
2	?	15.0	300.0	2.0	11.0	3.0	20.0	202.0	2.2	2.1
3	?	0.0	43.0	2.0	3.0	5.0	53.0	151.0	1.1	7.0

Comme vous pouvez le voir, le fichier contient 3 individus pour lesquels les 9 variables caractéristiques des légumes/fruits ont été spécifiées. Par contre, il ne contient pas de variable *classification* permettant de déterminer si une ligne concerne un fruit ou un légume. C'est cette variable que vous allez prédire grâce aux modèles entraînés précédemment.

En sortie de *File* avec lequel vous avez chargé les données à prédire, ajoutez un *Predictions* (menu *Evaluate*). En plus des données, ajoutez une connexion de façon à ce que *Predictions* prenne en entrée le modèle le plus performant (celui qui vous a donné la matrice ci-dessus) :



Ouvrez *prédictions*, vous pouvez observer que la première et la troisième ligne sont classées comme des fruits et que la deuxième est classée comme un légume :

	Logistic Regression	name	vitamin A %	vitamin C %	calcium %	iron %
1	fruit	?	1.0	154.0	3.0	1.0
2	vegetable	?	15.0	300.0	2.0	11.0
3	fruit	?	0.0	43.0	2.0	3.0

Ajoutez maintenant en entrée de *Predictions* tous les autres modèles :

	Logistic Regression	kNN	Naive Bayes	SVM	Tree	Random Forest	Neural Network
1	fruit	fruit	fruit	fruit	fruit	fruit	fruit
2	vegetable	fruit	vegetable	veget...	fruit	vegetable	vegetable
3	fruit	fruit	fruit	fruit	fruit	fruit	fruit

Comme vous pouvez le voir, tous classent la première et la dernière ligne comme des fruits. En revanche, l'algorithme des *k* plus proches voisins (*kNN*) et l'arbre de décision (*Tree*) classent la deuxième ligne comme un fruit alors que les autres la classent comme un légume.

Pour finir cet exercice, enregistrez votre travail (menu *File* -> *Save As ...*) pour le garder comme exemple.

Exercice 2

Lors de la deuxième séance, vous avez vu comment classer les textes d'un corpus à l'aide d'un algorithme de *clustering* hiérarchique. Comme vous l'avez vu en cours, ce type d'approche est qualifiée de non supervisée, *ie* on ne connaît pas les classes a priori et on se contente de regrouper les textes en fonction des mots qu'ils contiennent : plus les textes ont des mots en communs, plus ils ont de chances d'appartenir aux mêmes *clusters*. Vous avez vu aussi lors du TP 2 que :

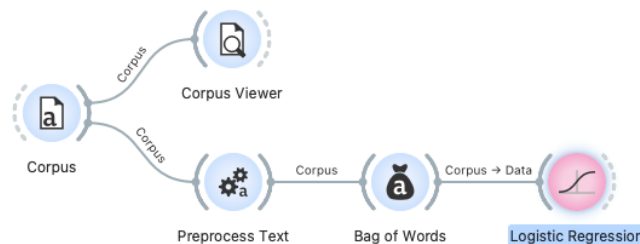
- (1) Les textes des contes de Grimm sont classés par l'ATU¹ en deux catégories, les contes d'animaux (*Animal Tales*) et les contes de magie (*Tales of Magic*),
- (2) L'algorithme de *clustering* hiérarchique a quasiment permis de retrouver ces deux classes.

L'objectif de cet exercice est de mettre en place une chaîne d'apprentissage supervisé pour apprendre, sur le corpus des contes de Grimm, à reconnaître les classes *Animal Tales* et *Tales of Magic*, puis classer les contes d'un autre corpus contenant des contes d'Andersen.

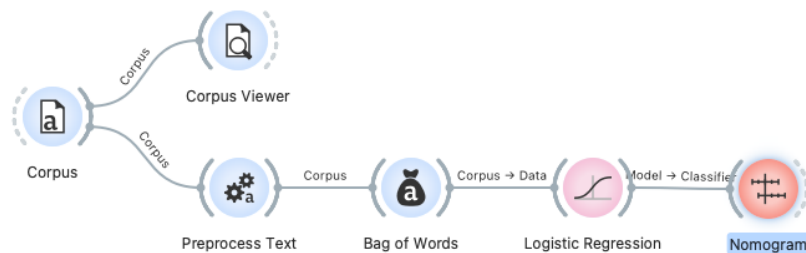
Ouvrez *Orange* et chargez *grimm-tales-selected.tab* à l'aide de *Corpus*. En sortie, utilisez *Corpus Viewer* pour voir les textes et vérifier qu'ils sont bien chargés.

Toujours en sortie de *Corpus*, ajoutez *Preprocess Text* (avec la liste des mots d'arrêt étendue comme vous l'avez fait lors du second TP) et *Bag of Words* pour prétraiter et transformer en nombres vos données.

En sortie de *Bag of Words*, ajoutez *Logistic Regression* pour apprendre à classer les données :

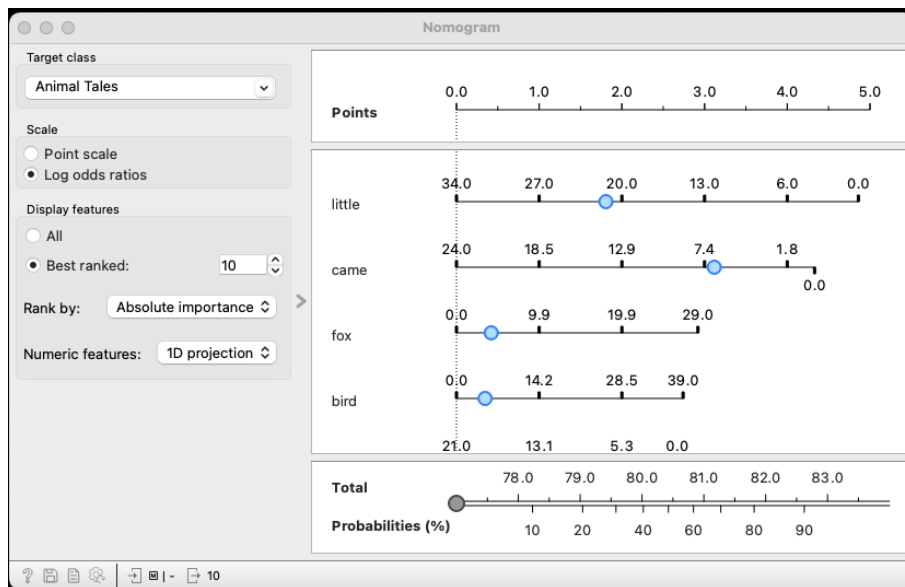


Pour visualiser comment le modèle de régression fonctionne, *ie* quels sont les mots qu'il utilise pour classer les textes (les mots discriminants), ajoutez l'outil *Nomogram* (menu *Visualize*) en sortie de *Logistic Regression* :

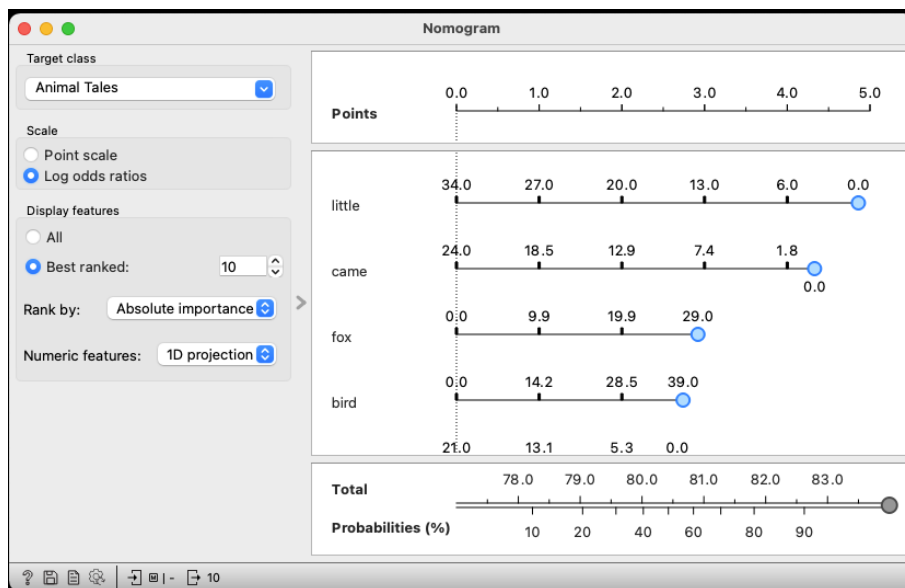


¹ https://fr.wikipedia.org/wiki/Classification_Aarne-Thompson-Uther

Un **nomogramme** est un outil permettant de visualiser comment les variables influencent les probabilités calculées par les modèles pour établir leur classification. Ouvrez *Nomogram* :

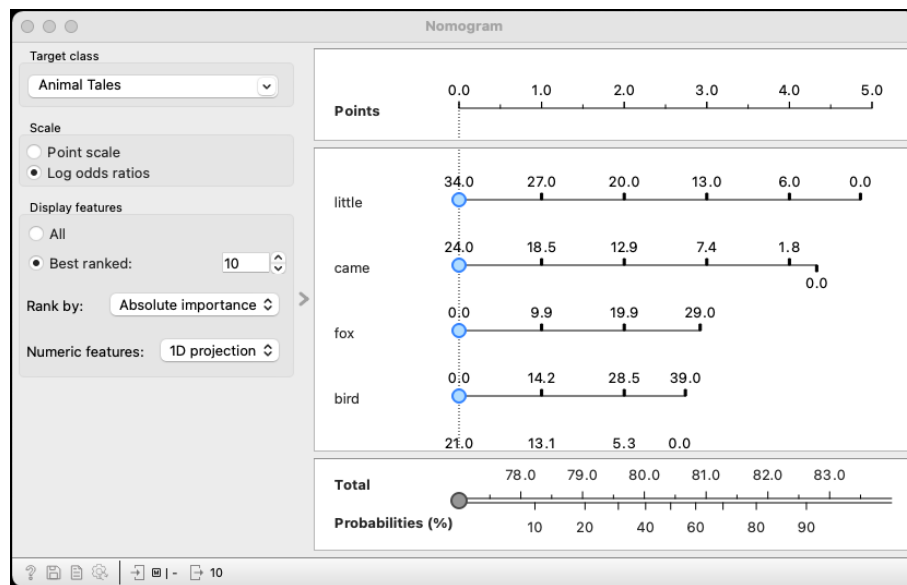


Dans le cadre de droite, vous pouvez voir la liste des 10 mots (*little*, *came*, *fox*...) les plus utilisés pour calculer la probabilité que le conte soit dans la classe *Animal Tales* (classe sélectionnée en haut à gauche). Placez tous les cercles bleus en fin de ligne :



Vous pouvez observer que le cercle gris en bas de l'écran est positionné à droite, cela veut dire que la probabilité que le conte soit dans la classe *Animal Tales* est de 100% si le nombre d'occurrences de *little* est 0, le nombre d'occurrences de *came* est 0, le nombre d'occurrences de *fox* est 29, le nombre d'occurrences de *bird* est 39, etc.

Mettez maintenant tous les cercles bleus à gauche :



Cette fois, vous observez que la probabilité que le texte soit dans la classe *Animal Tales* est de 0% si le nombre d'occurrences de *little* est 34, le nombre d'occurrences de *came* est 24, le nombre d'occurrences de *fox* est 0, le nombre d'occurrences de *bird* est 0, etc.

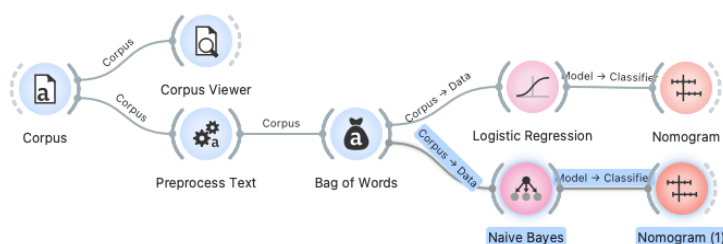
Déplacez les cercles bleus de façon à tester l'impact que les différents nombres d'occurrence des mots ont sur la probabilité que le texte soit dans la classe *Animal Tales*.

Vous pouvez remarquer que les échelles ne sont pas les mêmes pour chaque mot : parfois le 0 est à gauche, parfois il est à droite. Si le 0 est placé à gauche (par exemple pour le mot *fox*), cela indique que plus le mot apparaît dans le texte, plus la probabilité que ce texte soit dans la classe *Animal Tales* est élevée. Au contraire, si le 0 est placé à droite (par exemple pour le mot *little*), cela indique que moins le mot apparaît dans le texte, plus la probabilité que ce texte soit dans la classe *Animal Tales* est élevée.

Dans le cadre en haut à gauche, changez la classe ciblée en *Tales of Magic*. Vous pouvez observer que les axes sont inversés. Cette fois, plus le texte contient d'occurrences du mot *little*, plus la probabilité qu'il appartienne à la classe *Tales of Magic* est élevée. Au contraire, plus il contient d'occurrences du mot *fox*, plus la probabilité qu'il appartienne à la classe *Tales of Magic* est faible.

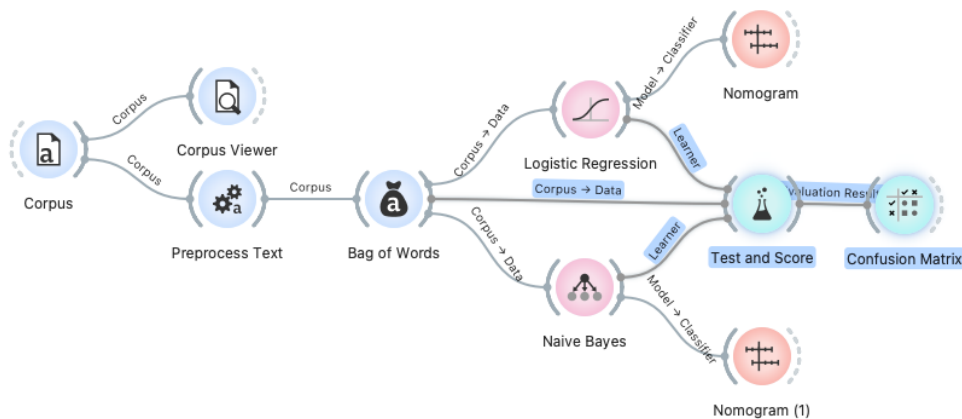
Continuez à déplacer les cercles bleus de façon à tester l'impact que les différents mots ont sur la probabilité que le texte soit dans la classe *Tales of Magic*.

L'outil *Nomogram* fonctionne aussi avec le modèle *Naive Bayes*. Ajoutez ce modèle à votre chaîne de traitement et comparez les deux nomogrammes issus des deux modèles :



Vous pouvez observer que certains mots sont parmi les 10 plus utilisés par les deux modèles, comme *little*, *went*, *came*, *cat*. D'autres en revanche n'apparaissent que dans le top 10 d'un seul modèle, comme *nothing*, *fox*...

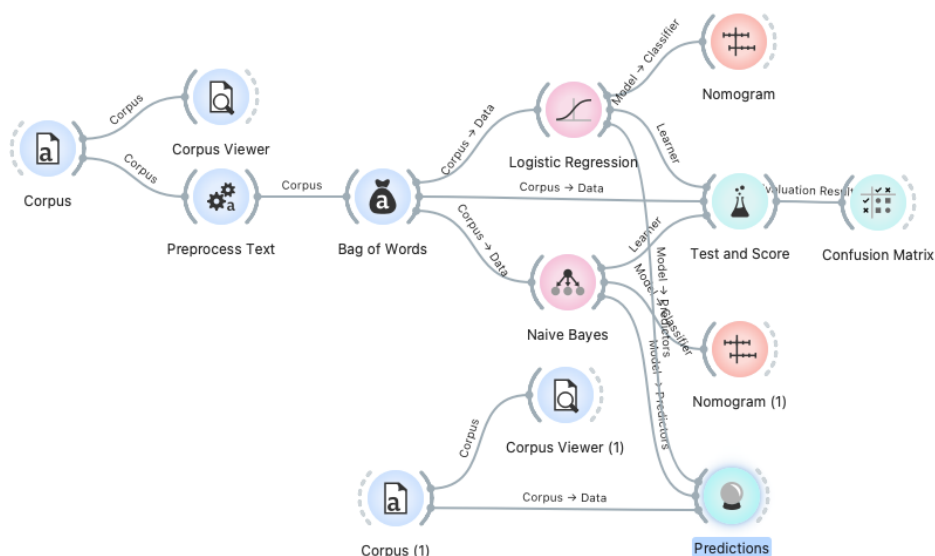
Vous allez maintenant essayer de voir si les deux modèles d'apprentissage sont performants. Pour cela, ajoutez *Test and Score* et *Confusion Matrix* en sortie des deux. N'oubliez pas que *Test and Score* nécessite aussi de prendre en entrée le jeu de donnée, en l'occurrence celui produit par le sac de mots :



Ouvrez *Confusion Matrix*. Vous pouvez observer que les deux modèles arrivent à retrouver tous les textes appartenant à la classe *Animal Tales*. Par contre, *Logistic Regression* classe quatre textes appartenant à la classe *Tales of Magic* dans la classe *Animal Tales* et *Naive Bayes* fait la même erreur pour 16 textes. La régression logistique semble donc être plus performante sur ce jeu de données.

Vous allez maintenant utiliser vos modèles pour classer des textes non étiquetés. Ajoutez un nouveau corpus et chargez-y le fichier *andersen.tab*. Utilisez l'outil *Corpus viewer* pour parcourir les 3 contes de ce nouveau corpus. Comme vous pouvez le remarquer, il n'y a pas de variable *ATU Topic* cette fois.

Ajoutez *Predictions* en sortie des deux modèles et connectez-le au nouveau corpus :



Ouvrez l'outil *Predictions* :

	Naive Bayes	Logistic Regression	Title	Content
1	Animal Tales	Tales of Magic	The Little Match-Seller	It was terribly cold and nearly dark on...
2	Tales of Magic	Tales of Magic	The Philosopher's Stone	Far away towards the east,in India,wh...
3	Tales of Magic	Tales of Magic	The Ugly Duckling	It was lovely summer weather in the ...

Vous pouvez observer que les trois contes ont été classés comme des *Tales of Magic* par la régression logistique. Cela paraît correct pour *The Little Match-Seller* (*La petite fille aux allumettes*²) et *The Philosopher's Stone* (*La pierre philosophale*³). En revanche, *The Ugly Duckling* (*Le vilain petit canard*⁴) devrait être classé comme *Animal Tales*. Le classifieur naïf de Bayes donne ici de moins bons résultats dans la mesure où deux contes semblent mal classés : *The Little Match-Seller* et *The Ugly Duckling*. Ces erreurs montrent qu'un modèle de classification n'est jamais sûr à 100% et qu'une vérification manuelle reste toujours nécessaire.

Pour finir cet exercice, enregistrez votre travail (menu *File -> Save As ...*) pour le garder comme exemple.

² Voir le résumé : https://fr.wikipedia.org/wiki/La_Petite_Fille_aux_allumettes

³ Texte en français : <https://www.biblisem.net/narratio/anderlpp.htm>

⁴ Voir le résumé : https://fr.wikipedia.org/wiki/Le_Vilain_Petit_Canard

Exercice 3

Dans les deux exercices précédents, vous avez vu des exemples de **classification binaire**, ie de classification avec 2 étiquettes. Il est aussi possible de faire des classifications avec plus d'étiquettes. On parle dans ce cas de **classification multiclass**.

Ouvrez *Orange* et chargez *iris.tab* avec *File*. Ajoutez en sortie de *File* un *MDS* et observez comment les données se répartissent dans le nuage de points résumant toutes les variables quantitatives. Vous pouvez observer que les iris de l'espèce *iris-setosa* se démarquent clairement des autres. Les deux autres espèces (*iris-versicolor* et *iris-virginica*) se démarquent aussi l'une de l'autre mais de manière moins tranchée.

Ajoutez en sortie de *File* les 7 modèles vus dans l'exercice 1. Ajoutez en sortie de ces modèles *Test and Score* et *Confusion Matrix*. Ouvrez *Confusion Matrix* et observez les différentes matrices de confusion. Comme on peut s'y attendre, les 50 individus de la classe *Iris-setosa* sont quasiment toujours bien classés. En revanche, les deux autres classes comportent plus d'erreurs (mais elles restent minimales).

Quel est le modèle le plus performant ? C'est plus difficile à dire ici, et ça le serait encore plus s'il y avait plus d'étiquettes. C'est pourquoi vous verrez la semaine prochaine comment quantifier, à travers différentes mesures, les performances d'un modèle.